

Gourika J Sreenivas

Problem Statement 2

Q1) Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]

LLM – B , DL – C , AI – B, ML - B

Q2) What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level

a) User Interface (UI) & Channel Layer

The frontend application captures the user input and displays the generated responses.

- **Web/Mobile Apps:** Custom widgets built with frameworks like React, Next.js, or Vue.js.
- **Messaging Platforms:** Third-party integrations like Slack, Microsoft Teams, WhatsApp, or Discord.
- **Prototyping UIs:** Fast, lightweight frameworks like [Streamlit](#) or Gradio used during development phases

b) Backend Application & Orchestration Layer

This acts as the central brain or middleware of the application, routing data between components and enforcing business logic

This acts as the central brain or middleware of the application, routing data between components and enforcing business logic.

- **API Gateway:** Manages incoming traffic, authenticates user tokens, and handles rate limiting
- **Orchestration Frameworks:** Software libraries such as [LangChain](#) or [LlamaIndex](#) that chain together prompts, models, and retrieval pipelines
- **Agent / Routing logic :** Analyzes intent to decide if a query needs standard text generation, data database searching, or external tool execution.

c) Context and memory management layer

LLMs are inherently stateless, meaning they treat every user input as a brand-new interaction. The memory system bridges this gap

Short-Term Memory: Temporarily buffers the immediate thread's history within a cache (e.g., Redis or local session state) so the LLM remembers previous messages in the current conversation

Long-Term Memory: Relational or NoSQL databases (e.g., PostgreSQL, MongoDB) that store permanent user profile data, historical interactions, and user preferences across multiple sessions

d) Retrieval-Augmented Generation (RAG) Layer

To eliminate hallucinations and ground your chatbot in domain-specific or corporate knowledge without fine-tuning, you must implement a RAG pipeline.

- **Document Ingestion (ETL):** Components that split raw text PDFs, internal wikis, or databases into optimized text chunks.

- **Embedding Model:** Converts text chunks and real-time user questions into mathematical vector representations
- **Vector Database:** Specialized data stores (such as Pinecone, Milvus, Qdrant, or Chroma) that run similarity searches to find the exact text chunks relevant to the user's query.

e) Core LLM & Tool Integration Layer

- This part is responsible for **understanding the user's request and generating the final response**.
- The application can use a hosted model such as GPT or Claude, or a self-hosted open-source model such as Llama or Mistral, depending on requirements around cost, privacy, latency, and infrastructure.
- The LLM does not necessarily have to perform every operation itself. Through **tool or function integration**, it can request actions from other systems.

For eg: if a user asks a question like what is my flight status , then that is passes to the LLM which then makes use of a flight-status api. The result is then returned back to the LLM and it generates a natural language response which is presented back to the user.

f) Guardrails & Safety Layer

- Since an LLM can generate incorrect, inappropriate, or potentially sensitive responses, the system should have controls around the mode.
- Input-side controls can check user requests for malicious instructions, prompt injection attempts, or sensitive information. Output-side controls can check whether the generated response violates application policies or exposes information that the user should not receive
- For an enterprise application, authorization is particularly important. A chatbot should not simply retrieve information because the LLM asks for it; the backend should verify whether the **user is actually permitted to access that information**.

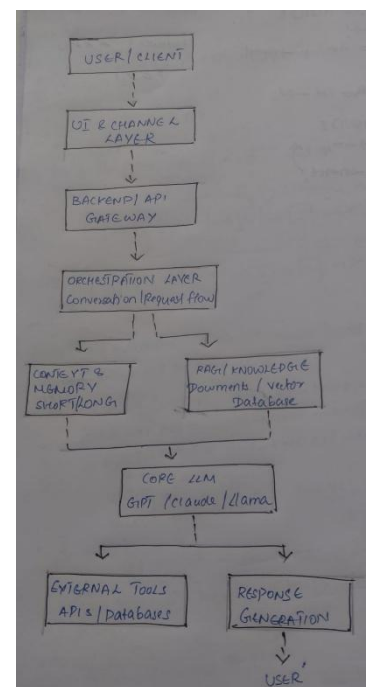
g) Observability & Monitoring Layer

The final layer is responsible for understanding how the chatbot behaves once it is deployed.

The system should monitor things such as:

- Response time and API latency
- Number of requests and failures
- LLM/token usage and cost
- Retrieval performance
- Errors in external tool calls
- User feedback and response quality

Tracing tool helps developers in understanding what happened during a Particular request , for eg , if the chatbot used database search , used the LLM directly, retrieved documents through RAG , or called an external API.



Q3) Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

A vector database is used to store **embeddings**, which are numerical representations of the meaning of data. Unlike a traditional database that mainly searches using exact values or keywords, a vector database can find information based on **semantic similarity** ie.. instead of matching exact keywords, it uses similarity math to find text, images or audio that refer to the same meaning.

Suppose I am building a placement preparation assistant for my college. The system would contain placement notices, company eligibility criteria, previous recruitment instructions, interview preparation material, and department-specific placement documents.

The database I would choose is Qdrant and my top reasons for choosing it would be:

- Semantic searching : students may ask the same question in different formats, so here similarity searching is much better than searching for exact matching keywords.
- **Metadata filtering:** Documents can be tagged with information such as company, department, placement year, or document type. This allows the search to be restricted before retrieving relevant content
- **Easy to integrate :** Qdrant can be easily integrated with python based RAG application without complicating the overall architecture
- **Metadata filtering:** Documents can be tagged with information such as company, department, placement year, or document type. This allows the search to be restricted before retrieving relevant content.